

*,È2 75Î1+ ¬2 7¤2 .ú 6l)5217(1' &+8<Ç1 6Æ8

*,È2 75Î1+ &66

7+,Â7.Â*, \$2',Ê1:(%

7ï1ÅQ 7§QJ /D\RXW ÿÃQ)OH[ER[*ULG &66 9DULDEOHV \$QLF

■ &66)RXQGDWLFQØH[ER[*ULG ■ 5HVSRQVLYH%(0 9DULDEO

7¬, /,Ê8 &+8<Ç1 6Æ8 7°3 7581* &66

7ËS /D7H; ÿÝF O±S WtFK KçS WRjQ EÝ F©x¹QuVjK J JyLL FKKmÇQL Ë Q
VDQJ PÑL WUuQK ELrQ GİFK

Ü, 1*É %,Ç1 62¤1 7(‰OLËX OmX KjQK QÝLEÝ

1 0

3KLrQ E§Q 6WDQGDORQH

&+,1+ 3+è& 1*+Ê7+8°7 7+,Â7.Â*, \$2',Ê1:(%+,Ê1 ¤,

Ü, 1*É %,Ç1 62▣1 :(%

7¬, /,Ê8 /I8 +¬1+ 1Ü, %Ü ± &+8<Ç1 Ä &66

*, È2 75Î1+ &66

7+, Â7.Â*, \$2', Ê1 :(%

7İ1ÅQ 7§QJ /D\RXW ŷÃQ)OH[ER[*ULG &66 9DULDEOHV

7°3 7581* %,Ç1 62▣1 7+ô& +¬1+ &66

%LrQ VR¥Q F&ILQKLIÃÑD WUõFÖTLXrDQ WÖF ŷÝ ELrQ GİFK

1 0

3KLrQ ESQ ;H/D7H; 8QLFRGH

[Khái Niệm] Ứng Dụng Thực Tế Của ::marker

- **Tạo danh sách đẹp mắt:** Dùng icon, biểu tượng mũi tên hoặc ký hiệu đặc biệt làm dấu đầu dòng.
- **Hướng dẫn từng bước:** Thay số thứ tự thành "Bước 1:", "Bước 2:", v.v.
- **Điểm nhấn thiết kế:** Làm nổi bật danh sách sản phẩm/tính năng mà không cần thêm thẻ HTML phức tạp.

8.7. Bảng Tổng Hợp Cheatsheet & Nguyên Tắc Thiết Kế Thực Tế**[Bảng] Bảng Cheatsheet Tra Cứu Toàn Diện 6 Pseudo-Element Selectors**

Pseudo-element	Chức năng chính	Thuộc tính hỗ trợ	Ứng dụng thực tế tiêu biểu
::before	Chèn nội dung trước phần tử	content, color, font, layout	Thêm icon, dấu hiệu, nhãn trang trí trước tiêu đề hoặc nút bấm.
::after	Chèn nội dung sau phần tử	content, color, font, layout	Thêm chú thích, hiệu ứng kết thúc, hoặc ký hiệu cảnh báo, clearfix.
::first-letter	Chọn chữ cái đầu tiên	font-size, color, float, margin	Tạo drop cap (chữ đầu lớn) nghệ thuật như trong sách, báo in.
::first-line	Chọn dòng đầu tiên	font, color, text-transform	Làm nổi bật dòng mở đầu bài viết blog, báo chí.
::selection	Chọn nội dung được bôi đen	background, color, text-shadow	Tạo hiệu ứng đặc biệt khi người dùng chọn văn bản.
::marker	Chọn ký hiệu danh sách	color, content, font-size	Tùy chỉnh bullet, số thứ tự trong danh sách .

[Khái Niệm] Tổng Kết & Ứng Dụng Thực Tế Nâng Cao

- **Tối ưu HTML:** Giảm số lượng thẻ HTML không cần thiết, giữ mã nguồn sạch sẽ và chuẩn W3C semantic.
- **Nâng cao UX/UI:** Tạo hiệu ứng tương tác thị giác đẹp mắt, tăng trải nghiệm người dùng khi đọc và tương tác.
- **Dễ bảo trì mã nguồn:** Chỉ cần thay đổi CSS trong một file duy nhất mà không cần sửa cấu trúc mã HTML hàng loạt.

Ví dụ thực tiễn: Khi làm menu điều hướng hoặc card sản phẩm, bạn có thể dùng `::before` để thêm biểu tượng icon nhỏ, hoặc `::selection` để tạo trải nghiệm đọc thương hiệu thú vị hơn.

8.8. Đọc Thêm: Chuyên Đề Nâng Cao Về Pseudo-Elements (Mở Rộng Kiến Thức Thực Chiến)**[Khái Niệm] 1. Các Giá Trị Đa Dạng Của Thuộc Tính content Trong ::before / ::after**

Thuộc tính `content` trong `::before` và `::after` không chỉ nhận chuỗi văn bản đơn thuần, mà còn hỗ trợ nhiều kiểu giá trị cực kỳ mạnh mẽ:

- `content: "Chuoi van ban";`: Chèn văn bản trực tiếp.
- `content: "";`: Chuỗi rỗng (Thường kết hợp với `display: block/inline-block`, `width`, `height`, `background` để vẽ các hình khối trang trí, đường kẻ, hoặc overlay).
- `content: attr(attribute-name);`: Lấy giá trị thuộc tính HTML động. Rất phổ biến để tạo Tooltip hoặc hiển thị đường dẫn URL khi in trang web.
- `content: url(image.png);`: Chèn hình ảnh trực tiếp.
- `content: counter(name);`: Tạo bộ đếm số tự động trong CSS (CSS Counters).

●●● Ví Dụ Ứng Dụng content: attr(data-tooltip) Để Tạo Tooltip Thuần CSS

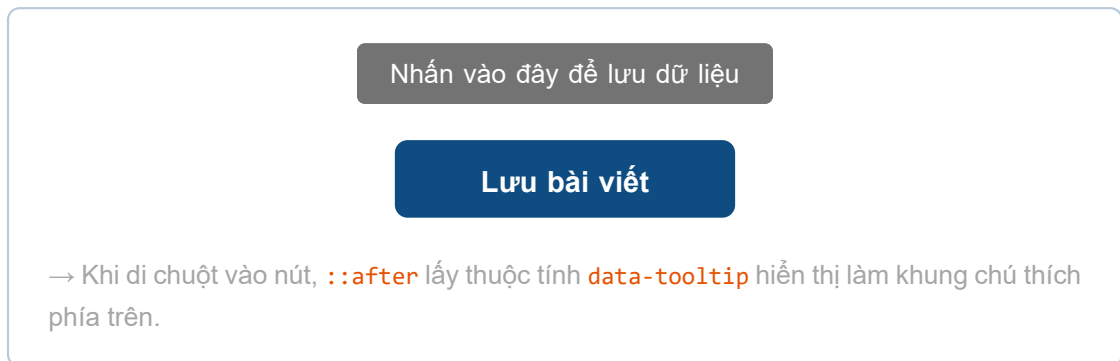
```

1 <button class="btn" data-tooltip="Nhấn vào đây để lưu dữ liệu">Lưu bài viết<
  /button>
2
3 <style>
4   .btn { position: relative; }
5   .btn:hover::after {
6     content: attr(data-tooltip);
7     position: absolute;
8     bottom: 100%;
9     left: 50%;
10    transform: translateX(-50%);
11    background: #333;
12    color: #fff;
13    padding: 4px 8px;
14    border-radius: 4px;
15    white-space: nowrap;
16    font-size: 12px;
17  }
18 </style>

```

●●● BROWSER PREVIEW

Kết Quả: Tooltip Động Thuần CSS Với content: attr()



[Bảng] 2. Bảng So Sánh Chi Tiết: Pseudo-class (:) vs Pseudo-element (::)

Tiêu chí	Pseudo-class (:)	Pseudo-element (::)
Cú pháp tiêu chuẩn CSS3	Sử dụng 1 dấu hai chấm (: hover)	Sử dụng 2 dấu hai chấm (:: before)
Bản chất tác động	Chọn phần tử dựa trên trạng thái tương tác hoặc vị trí vị thế DOM .	Định kiểu cho một phần nội dung cụ thể hoặc tạo phần tử ảo .
Sự tồn tại trong DOM	Nhắm trực tiếp vào thẻ HTML thực tế đã có trong cây DOM.	Tạo phần tử ảo hiển thị trên màn hình nhưng KHÔNG tồn tại trong cây DOM.
Ví dụ tiêu biểu	:hover , :focus , :nth-child(2)	::before , ::after , ::first-letter

[Cảnh Báo] 3. Lưu Ý Quan Trọng Về Accessibility (Khả Năng Truy Cập - a11y) & Hiệu Năng

- **Công cụ đọc màn hình (Screen Readers):** Một số công cụ đọc màn hình có thể đọc nội dung trong thuộc tính `content: "..."` của `::before` và `::after`. Tránh chèn các ký tự trang trí không có nghĩa ngữ văn bản (ví dụ: `content: ">>> ";`) vì có thể gây phiền cho người khiếm thị. Nếu nội dung chỉ mang tính trang trí, hãy cân nhắc áp dụng thuộc tính `aria-hidden="true"`.
- **Modern CSS Syntax for Alt Text:** Trong CSS Modern, bạn có thể truyền văn bản thay thế alt trực tiếp trong `content: "STAR" / "Bieu tuong ngoi sao";`.
- **Hiệu năng Rendering:** Các pseudo-elements được xử lý rất nhanh bởi GPU của trình duyệt. Tuy nhiên khi kết hợp animation biến đổi (`transform`, `opacity`) trên `::before/::after`, hãy luôn ưu tiên sử dụng thuộc tính `will-change` để tránh hiện tượng giật khung hình (**Jank**).

[Mẹo] 4. Bộ Câu Hỏi Phỏng Vấn Tuyển Dụng Frontend Chuyên Sâu (Interview Q&A)**1. Q1: Phân biệt sự khác biệt cốt lõi giữa 1 dấu hai chấm (:) và 2 dấu hai chấm (::) trong CSS?**

→ **Trả lời ghi điểm:** Dấu single colon (:) dùng cho Pseudo-classes đại diện cho trạng thái tương tác của phần tử có sẵn trong DOM (như `:hover`, `:focus`, `:nth-child`). Dấu double colon (::) dùng cho Pseudo-elements đại diện cho phần tử ảo không có trong DOM hoặc một phần nội dung cụ thể (như `::before`, `::after`, `::first-letter`).

**2. Q2: Tại sao không thể sử dụng ::before hay ::after trên các thẻ , <input>,
?**

→ **Trả lời ghi điểm:** Vì các thẻ như `<img>`, `<input>`, `<br>` là các phần tử thay thế (**Replaced Elements** / **Void Elements**). Chúng không chứa phần nội dung con bên trong (no child content), do đó trình duyệt không có vị trí con đầu tiên (first child) hay con cuối cùng (last child) để chèn phần tử giả `::before` hay `::after`.

3. Q3: Điều gì xảy ra nếu bạn quên thuộc tính content khi khai báo ::before?

→ **Trả lời ghi điểm:** Thuộc tính `content` là điều kiện tiên quyết bắt buộc. Nếu không có `content` (hoặc nếu set `content: none`), trình duyệt sẽ bỏ qua và không khởi tạo phần tử giả đó trên cây Render Tree.

4. Q4: Kỹ thuật Micro clearfix giải quyết vấn đề gì bằng ::after?

→ **Trả lời ghi điểm:** Giải quyết hiện tượng sụp đổ chiều cao (**Height Collapse**) của thẻ cha khi các thẻ con dùng `float`. Bằng cách chèn `::after` có `content: ""; display: table; clear: both;`, thẻ cha tự động bao bọc trọn vẹn toàn bộ chiều cao của các thẻ con float mà không cần chèn thẻ HTML rỗng.

5. Q5: Làm thế nào để đặt một phần tử giả ::before nằm phía sau background của thẻ cha?

→ **Trả lời ghi điểm:** Thiết lập `position: relative` và `z-index: 1` (hoặc `isolation: isolate`) trên thẻ cha để tạo một Stacking Context độc lập, sau đó thiết lập `position: absolute` và `z-index: -1` trên `::before`.

8.9. Phân Tích & Xử Lý Xung Đột Khi Sử Dụng Pseudo-elements (CSS Conflicts Masterclass)

Khi phát triển giao diện Web phức tạp, việc khai báo nhiều quy tắc CSS cho cùng một bộ chọn phần tử giả có thể gây ra xung đột thị giác. Dưới đây là phân tích chi tiết 4 dạng xung đột phổ biến nhất cùng giải pháp xử lý triệt để:

[Khái Niệm] 1. Xung Đột Khi Khai Báo Nhiều Lần Cho Cùng Một Pseudo-element

Nguyên lý duy nhất (Single Instance Rule): Mỗi phần tử HTML chỉ sở hữu duy nhất **một instance (thể hiện ảo)** của từng loại phần tử giả (`::before` hoặc `::after`).

Cơ chế xung đột (Cascade Override): Nếu bạn khai báo nhiều khối CSS cùng nhắm vào `.class::before`, trình duyệt sẽ áp dụng quy tắc Cascading mặc định: **Quy tắc xuất hiện sau cùng trong mã nguồn sẽ ghi đè hoàn toàn quy tắc trước đó**!

●●● Ví Dụ Xung Đột Khai Báo Nhiều Khối ::before

```

1 <p class="text">Chao ban!</p>
2
3 <style>
4   /* Quy tắc 1: Bi ghi de */
5   .text::before {
6       content: "Xin chao! ";
7       color: red;
8   }
9   /* Quy tắc 2: Chien thang (viet sau) */
10  .text::before {
11      content: "Hello! ";
12      color: blue;
13  }
14 </style>

```

●●● BROWSER PREVIEW Kết Quả Hiện Thị: Quy Tắc Viết Sau Ghi Đè Quy Tắc Trước

Kết quả: **Hello!** Chào bạn!

Giải thích: Quy tắc thứ hai (.text::before có content: "Hello! ") được viết sau nên ghi đè quy tắc thứ nhất.

[Mẹo] Giải Pháp Xử Lý Xung Đột Khai Báo Trùng

Giải pháp: Gộp tất cả thuộc tính định kiểu vào một khối CSS duy nhất hoặc sử dụng các selector có độ đặc hiệu (Specificity) cao hơn để phân biệt ngữ cảnh rõ ràng:

●●● Mã Nguồn Giải Pháp Gộp Style Chuẩn

```

1 .text::before {
2   content: "Xin chao! ";
3   color: red;
4   font-weight: bold;
5 }

```

[Khái Niệm] 2. Xung Đột Khi Thẻ Cha Mang Thuộc Tính display: none

Hiện tượng: `::before` và `::after` tạo ra nội dung ảo nằm ****bên trong**** phần tử cha trên Render Tree.

Hệ quả: Nếu thẻ cha bị ẩn bằng thuộc tính `display: none`, toàn bộ phần tử giả con bên trong cũng sẽ ****biến mất 100%**** khỏi màn hình hiển thị, bất kể bạn có cố gắng đặt `display: block` hay `visibility: visible` riêng cho phần tử giả!

●●● Ví Dụ Thẻ Cha display: none Làm Biến Mất Pseudo-element

```

1 <p class="hidden-box">Nội dung bài viết</p>
2
3 <style>
4   p.hidden-box {
5     display: none; /* An toàn bỏ thẻ cha */
6   }
7   p.hidden-box::before {
8     content: "Không thay tôi đau!";
9     color: red;
10  }
11 </style>

```

●●● BROWSER PREVIEW**Kết Quả Hiển Thị: Không Có Gì Hiển Thị**

(Không hiển thị bất kỳ thành phần nào trên màn hình vì thẻ cha p bị ẩn hoàn toàn bởi display: none)

[Mẹo] Giải Pháp Khi Cần Ẩn Nội Dung Thật Nhưng Giữ Phần Tử Giả

Giải pháp: Nếu muốn ẩn nội dung thật của thẻ cha mà vẫn duy trì phần tử giả hiển thị, bạn có thể thiết lập `font-size: 0` trên thẻ cha và đặt lại `font-size` chuẩn trên phần tử giả, hoặc dùng thuộc tính `visibility: hidden` kết hợp `visibility: visible` trên phần tử giả!

[Khái Niệm] 3. Xung Đột Tương Tác Giữa ::first-letter / ::first-line Với ::before / ::after

Nguyên lý tương tác: Bộ chọn `::first-letter` và `::first-line` được thiết kế để định dạng ****văn bản thực tế**** trong thẻ HTML. Chúng ****KHÔNG**** tự động tác động lên ký tự hoặc nội dung ảo được chèn từ thuộc tính `content` của `::before`!

●●● Ví Dụ ::first-letter Không Ảnh Hưởng Đến Nội Dung Ảo Từ ::before

```

1 <p class="intro">Chao mung ban!</p>
2
3 <style>
4   p.intro::before {
5     content: "STAR "; /* Noi dung ao */
6   }
7   p.intro::first-letter {
8     color: red;
9     font-size: 30px;
10  }
11 </style>

```

●●● BROWSER PREVIEW

Kết Quả Hiển Thị: ::first-letter Định Kiểu Cho Chữ C

STAR **C**ào mừng bạn!

Giải thích: ::first-letter tác động vào chữ cái đầu "C" của nội dung thật, không phải chữ "S" trong nội dung giả "STAR".

[Mẹo] Giải Pháp Định Kiểu Riêng Cho Nội Dung Giả

Giải pháp: Nếu muốn định dạng kiểu chữ, màu sắc hay kích thước cho nội dung giả từ **::before**, hãy trực tiếp khai báo các thuộc tính đó bên trong khối **::before**:

●●● Khai Báo Định Kiểu Trực Tiếp Trong ::before

```

1 p.intro::before {
2   content: "STAR ";
3   color: green;
4   font-size: 30px;
5   font-weight: bold;
6 }

```

[Khái Niệm] 4. Xung Đột Thứ Tự CSS Về Độ Ưu Tiên (Specificity Conflict)

Khi hai bộ chọn cùng nhắm tới phần tử giả của một thẻ (ví dụ: **h1::before** và **h1.special::before**), trình duyệt sẽ tính toán điểm ****CSS Specificity**** để quyết định quy tắc nào chiến thắng.

 Ví Dụ Bộ Chọn Cụ Thể Hơn Chiến Thắng

```
1 <h1 class="special">Tieu de trang web</h1>
2
3 <style>
4   /* Specificity: (0,0,0,2) - Class va Element */
5   h1.special::before {
6     content: "[SPECIAL] ";
7     color: blue;
8   }
9
10  /* Specificity: (0,0,0,1) - Element */
11  h1::before {
12    content: "[DEFAULT] ";
13    color: orange;
14  }
15 </style>
```

 BROWSER PREVIEW

Kết Quả Hiển Thị: Specificity Cao Hơn Chiến Thắng

[SPECIAL] Tiêu đề trang web

Giải thích: Thẻ h1 có class "special" nên bộ chọn h1.special::before (điểm cao hơn) chiến thắng.

[Bảng] Bảng Tra Cứu Toàn Diện Các Dạng Xung Đột, Nguyên Nhân & Giải Pháp Xử Lý

Pseudo-element	Bản chất đối tượng	Nguyên nhân xung đột thường gặp	Giải pháp xử lý chuẩn mực
<code>::before / ::after</code>	Nội dung ảo chèn vào đầu/cuối phần tử	Trùng lặp khai báo nhiều khối CSS; bị ẩn khi thẻ cha dùng <code>display: none</code> .	Dùng selector cụ thể hơn để phân tách; điều chỉnh logic hiển thị bằng <code>visibility</code> .
<code>::first-letter</code>	Chữ cái đầu tiên của văn bản thật	Không tự động tác động lên nội dung ảo của <code>::before</code> .	Khai báo định kiểu font/color trực tiếp trong khối <code>::before</code> .
<code>::first-line</code>	Dòng đầu tiên của văn bản khối	Không áp dụng được các thuộc tính layout như <code>margin</code> , <code>padding</code> , <code>width</code> .	Chỉ dùng các thuộc tính văn bản/màu sắc hợp lệ (<code>color</code> , <code>font</code> , <code>line-height</code>).
Thứ tự CSS (Cascade)	Quy tắc ghi nguồn từ trên xuống dưới	Khai báo sau ghi đè khai báo trước nếu cùng Specificity.	Kiểm tra thứ tự nạp file CSS, gộp thuộc tính trùng lặp.
Độ ưu tiên (Specificity)	Điểm số bộ chọn trình duyệt tính toán	Class mạnh hơn Element, ID mạnh hơn Class.	Nắm vững công thức Specificity; hạn chế tối đa việc lạm dụng <code>!important</code> .

[Mẹo] 3 Lời Khuyên Thực Chiến Tối Ưu Quản Lý Pseudo-elements

- Viết CSS rõ ràng, mô-đun hóa:** Đừng chồng chéo quá nhiều phần tử giả không cần thiết trên cùng một thẻ HTML.
- Kiểm tra điểm Specificity:** Khi thấy phần tử giả không hiển thị đúng màu hoặc content, hãy kiểm tra xem có selector nào mạnh hơn đang ghi đè hay không.
- Thành thạo công cụ Chrome DevTools:** Mở tab *Elements* trên trình duyệt, click mở rộng thẻ HTML target để soi trực tiếp phần tử ảo `::before / ::after` và kiểm tra các dòng CSS đang bị gạch ngang (ghi đè).

1.7 Thứ Tự Ưu Tiên Trong CSS (CSS Specificity Masterclass)

Trong thiết kế giao diện Web thực tế, một phần tử HTML thường nằm trong phạm vi ảnh hưởng của nhiều quy tắc CSS khác nhau (ví dụ: vừa có style từ thẻ `p`, vừa có class `.text`, vừa có ID `#main`, lại vừa có thuộc tính `style="..."` trực tiếp). Khi xảy ra xung đột giữa các quy tắc này, trình duyệt sẽ dùng cơ chế **CSS Specificity** (Thứ tự ưu tiên / Tính đặc hiệu) để tính toán điểm số và quyết định quy tắc nào sẽ chiến thắng.

[Khái Niệm] Khái Niệm Specificity (Tính Đặc Hiệu) Là Gì?

CSS Specificity (Tính đặc hiệu) là thuật toán và hệ thống quy tắc mà trình duyệt sử dụng để quyết định quy tắc CSS nào sẽ được áp dụng cho một phần tử HTML khi có nhiều quy tắc cùng nhắm vào phần tử đó.

Vì sao cần Specificity? Khi trang web phát triển lớn, một phần tử (ví dụ: nút bấm `<button id="submit" class="btn">`) thường khớp với nhiều quy tắc CSS khác nhau. Specificity giúp trình duyệt tính toán điểm số công bằng để chọn ra quy tắc mạnh nhất dựa trên độ cụ thể của bộ chọn, **không phụ thuộc vào thứ tự viết code** (trừ khi hai bộ chọn có điểm số bằng nhau).

1.7.1 1. Bảng Xếp Hạng Chi Tiết Độ Ưu Tiên Specificity (6 Bậc Giá Trị)

[Bảng] Bảng Xếp Hạng Chi Tiết Độ Ưu Tiên Trong CSS				
Cấp độ	Loại bộ chọn (Selector)	Cú pháp minh họa	Specificity	Đặc tính & Độ mạnh
(1)	!important trong CSS	<code>color: red !important;</code>	—	[Cao nhất] – Ghi đè tất cả (trừ inline style có !important)
(2)	Style nội tuyến (Inline style)	<code><div style="color:red;"></code>	(1,0,0,0)	Nội tuyến – Rất mạnh, ghi đè mọi ID và Class
(3)	Bộ chọn ID (#id)	<code>#header { color: blue; }</code>	(0,1,0,0)	ID Selector – Rất mạnh, nhưng hạn chế lạm dụng
(4)	Class, Attribute, Pseudo-class	<code>.btn, [type="text"], :hover</code>	(0,0,1,0)	Chung nhóm – Linh hoạt và được sử dụng phổ biến nhất
(5)	Thẻ HTML & Pseudo-element	<code>div, h1, p, ::before</code>	(0,0,0,1)	Thẻ Element – Mức độ yếu nhất trong các bộ chọn
(6)	Bộ chọn chung & Kế thừa	<code>* { margin: 0; }, body { color }</code>	—	Mặc định – Yếu nhất, dễ dàng bị bất kỳ bộ chọn nào ghi đè

1.7.2 2. Quy Trình 4 Bước Duyệt DOM & Công Thức (A, B, C, D) / (a, b, c, d)

Trình duyệt thực hiện quá trình tính toán và áp dụng kiểu dáng theo 4 bước tiêu chuẩn:

[Khái Niệm] 4 Bước Xử Lý Thứ Tự Ưu Tiên Của Trình Duyệt**Bước 1: Thu thập tất cả các quy tắc CSS áp dụng cho phần tử.**

Ví dụ: Với thẻ `<p id="main" class="text">`, trình duyệt gom tất cả các quy tắc khớp với phần tử này như `p`, `.text`, `#main`, `p.text`, `#main.text...`

Bước 2: Tính điểm specificity cho từng bộ chọn theo dạng 4 nhóm (A, B, C, D) hoặc (a, b, c, d).

- **Nhóm A / a (1,0,0,0):** Style nội tuyến (Inline styles) đặt trực tiếp trong thuộc tính `style="..."`.
- **Nhóm B / b (0,1,0,0):** Số lượng ID selectors (`#id`).
- **Nhóm C / c (0,0,1,0):** Số lượng Class (`.text`), Attribute (`[type="text"]`), Pseudo-class (`:hover`, `:focus`, `:not()`).
- **Nhóm D / d (0,0,0,1):** Số lượng Element (thẻ HTML như `div`, `p`) và Pseudo-element (`::before`, `::after`).

Bước 3: So sánh specificity theo thứ tự ưu tiên $A > B > C > D$ (hoặc $a > b > c > d$).

Trình duyệt so sánh từng chỉ số từ trái sang phải (giống như so sánh số nguyên nhiều chữ số):

- `p` → Specificity = (0,0,0,1)
- `.text` → Specificity = (0,0,1,0)
- `#main` → Specificity = (0,1,0,0)
- `style="..."` → Specificity = (1,0,0,0)
- `p.text:hover` → Specificity = (0,0,2,1)

Bước 4: Áp dụng quy tắc có specificity cao nhất.

- Nếu nhiều quy tắc có **cùng điểm specificity**, quy tắc nào **viết sau trong file CSS sẽ thắng** (theo thứ tự xuất hiện **Source Order**).
- Nếu có từ khóa **!important**, trình duyệt sẽ cấp quyền ưu tiên tuyệt đối để đè qua tất cả chỉ số specificity thông thường.

[Bảng] Bảng Định Nghĩa 4 Thành Phần Của Công Thức (A, B, C, D) / (a, b, c, d)

Ký hiệu	Thành phần quy đổi	Giá trị mẫu	Quy tắc cộng điểm
A / a	Style nội tuyến (Inline style)	(1,0,0,0)	+1 nếu phần tử được viết trực tiếp trong thuộc tính <code>style="..."</code>
B / b	Số lượng ID selectors (#id)	(0,1,0,0)	+1 cho mỗi ID xuất hiện trong bộ chọn
C / c	Class, Attribute, Pseudo-classes	(0,0,1,0)	+1 cho mỗi <code>.class</code> , <code>[attr]</code> , <code>:hover</code> , <code>:nth-child()</code>
D / d	Thẻ HTML (Elements) & Pseudo-elements	(0,0,0,1)	+1 cho mỗi thẻ HTML (<code>div</code> , <code>p</code>) và <code>::before</code> , <code>::after</code>

[Ghi Chú] Lưu Ý Quan Trọng Khi Tính Điểm Specificity

- **Cách so sánh:** Trình duyệt so sánh từ trái sang phải: *A* lớn hơn thì thắng; nếu *A* bằng nhau thì so sánh *B*; nếu *B* bằng nhau thì so sánh *C*; nếu *C* bằng nhau thì so sánh *D*.
- **Không quy đổi thập phân:** Điểm (0, 0, 1, 0) đại diện cho 1 Class luôn luôn mạnh hơn bất kỳ số lượng thẻ HTML nào (ví dụ (0, 0, 0, 11) gồm 11 thẻ HTML vẫn thua 1 Class).
- **Universal Selector (*) và Combinators (>, +, ~):** Có điểm số là (0, 0, 0, 0), không cộng thêm điểm specificity.
- **File Internal (<style>) vs External (.css):** Vị trí khai báo CSS không làm thay đổi specificity, chỉ bị ảnh hưởng bởi specificity và thứ tự xuất hiện.

1.7.3 3. Bảng Tra Cứu Specificity Của Các Bộ Chọn Thường Gặp (15 Ví Dụ)

[Bảng] Bảng Tra Cứu Specificity Của Các Bộ Chọn CSS Phổ Biến (Tự Động Sắp Xếp Từ Thấp Đến Cao)

Bộ chọn CSS (Selector)	Specificity (A,B,C,D)	Phân tích chi tiết thành phần
<code>h1</code>	(0,0,0,1)	1 thẻ HTML (<code>h1</code>)
<code>div</code>	(0,0,0,1)	1 thẻ HTML (<code>div</code>)
<code>.box / .btn</code>	(0,0,1,0)	1 class (<code>.box</code> hoặc <code>.btn</code>)
<code>#header / #main</code>	(0,1,0,0)	1 ID (<code>#header</code> hoặc <code>#main</code>)
<code>div p</code>	(0,0,0,2)	2 thẻ HTML (<code>div</code> + <code>p</code>)
<code>p::first-line</code>	(0,0,0,2)	1 thẻ HTML (<code>p</code>) + 1 pseudo-element (<code>::first-line</code>)
<code>a:hover</code>	(0,0,1,1)	1 thẻ HTML (<code>a</code>) + 1 pseudo-class (<code>:hover</code>)
<code>input[type="text"]</code>	(0,0,1,1)	1 thẻ HTML (<code>input</code>) + 1 attribute (<code>[type]</code>)
<code>div:not(.active)</code>	(0,0,1,1)	1 thẻ HTML (<code>div</code>) + 1 class bên trong <code>:not()</code> (<code>.active</code>)
<code>.nav ul li a</code>	(0,0,1,3)	1 class (<code>.nav</code>) + 3 thẻ HTML (<code>ul</code> , <code>li</code> , <code>a</code>)
<code>#main .button / div#main .btn:hover</code>	(0,1,2,1)	1 ID (<code>#main</code>) + 2 class (<code>.btn</code> , <code>:hover</code>) + 1 thẻ (<code>div</code>)
<code>a[href][data-id].active:hover</code>	(0,0,4,1)	1 thẻ (<code>a</code>) + 2 attributes (<code>[href]</code> , <code>[data-id]</code>) + 2 class (<code>.active</code> , <code>:hover</code>)
<code>section#hero .content p</code>	(0,1,1,2)	1 ID (<code>#hero</code>) + 1 class (<code>.content</code>) + 2 thẻ (<code>section</code> , <code>p</code>)
<code>html body .wrapper #main-content p</code>	(0,1,1,3)	1 ID (<code>#main-content</code>) + 1 class (<code>.wrapper</code>) + 3 thẻ (<code>html</code> , <code>body</code> , <code>p</code>)
<code>style="color:red" (inline style)</code>	(1,0,0,0)	1 thuộc tính nội tuyến <code>style="..."</code>

1.7.4 4. Những Quy Tắc Đặc Biệt Trong Specificity

4.1. Từ Khóa !important Không Được Tính Vào Specificity Nhưng Ghi Đều Tất Cả

[Khái Niệm] Tại Sao !important Không Tính Vào Specificity?

!important là một quy tắc ưu tiên tuyệt đối trong thuật toán Cascade của trình duyệt, nhưng nó **không được cộng điểm vào bộ số Specificity** (A, B, C, D). Điều này có nghĩa là, dù một quy tắc có specificity thấp, nếu có **!important**, nó vẫn được áp dụng thay vì các quy tắc có specificity cao hơn nhưng không có **!important**.

●●● Ví Dụ 1: Quy Tắc Specificity Cao Hơn Nhưng Không Có !important

```

1 /* Specificity: (0,1,0,1) - Không có !important */
2 #content p {
3   color: red;
4 }
5
6 /* Specificity: (0,0,0,1) - Thấp hơn nhưng CÓ !important -> WINS! */
7 p {
8   color: blue !important;
9 }

```

●●● BROWSER PREVIEW

Kết Quả Ví Dụ 1: Văn Bản <p> Có Màu Xanh (Blue)

Đoạn văn hiển thị màu blue

Giải thích: #content p có specificity (0,1,0,1), p có specificity (0,0,0,1) thấp hơn. Nhưng nhờ từ khóa !important, quy tắc p { color: blue !important; } ghi đè tất cả và màu xanh (blue) chiến thắng.

●●● Ví Dụ 2: Nếu Cả Hai Quy Tắc Đều Có !important

```

1 /* Specificity: (0,1,0,1) + !important -> WINS! */
2 #content p {
3   color: red !important;
4 }
5
6 /* Specificity: (0,0,0,1) + !important -> LOSES! */
7 p {
8   color: blue !important;
9 }

```

●●● BROWSER PREVIEW

Kết Quả Ví Dụ 2: Văn Bản <p> Có Màu Đỏ (Red)

Đoạn văn hiển thị màu red

Giải thích: Khi cả hai quy tắc đều có !important, quyền ưu tiên tuyệt đối triệt tiêu lẫn nhau. Trình duyệt sẽ quay lại so sánh specificity: #content p có (0,1,0,1) cao hơn p (0,0,0,1), nên màu đỏ (red) chiến thắng.

4.2. Universal Selector (*) Và Combinators (>, +, ~) Không Ảnh Hưởng Đến Specificity

[Khái Niệm] Tại Sao *, >, +, ~ Khóa Ảnh Hưởng Đến Specificity?

- **Universal selector (*)**: Là bộ chọn chung áp dụng cho mọi phần tử, nhưng không có giá trị specificity (0, 0, 0, 0) vì nó không giúp phân biệt một phần tử cụ thể.
- **Combinators (>, +, ~)**: Chỉ định quan hệ giữa các phần tử (con trực tiếp, anh em liền kề, anh em chung cha), nhưng không làm tăng specificity.

●●● Ví Dụ 3: Universal Selector (*)

```
1 /* Specificity: (0,0,0,0) */
2 * {
3   color: green;
4 }
5
6 /* Specificity: (0,0,0,1) - WINS! */
7 p {
8   color: red;
9 }
```

●●● BROWSER PREVIEW

Kết Quả Ví Dụ 3: Thẻ <p> Có Màu Đỏ (Red)

Đoạn văn hiển thị màu red

Giải thích: * { color: green; } có specificity (0,0,0,0), p { color: red; } có specificity (0,0,0,1) cao hơn nên màu đỏ chiến thắng.

●●● Ví Dụ 4: Combinator (>) Không Làm Tăng Specificity

```
1 /* Specificity: (0,0,0,2) - 2 elements (div, p), combinator > có điểm = 0 */
2 div > p {
3   color: blue;
4 }
5
6 /* Specificity: (0,0,0,1) - 1 element */
7 p {
8   color: red;
9 }
```


**BROWSER PREVIEW****Kết Quả Ví Dụ 4: `div > p` (0,0,0,2) Thắng `p` (0,0,0,1)****Đoạn văn trong `div` có màu blue**

Giải thích: `div > p` gồm 2 thẻ element (`div` và `p`) nên có specificity (0,0,0,2) cao hơn `p` (0,0,0,1). Bản thân dấu `>` có điểm = 0.

4.3. CSS Internal (<style>) Và External (.css File) Không Ảnh Hưởng Độ Ưu Tiên

[Khái Niệm] Tại Sao Cách Viết CSS (Internal Hay External) Không Ảnh Hưởng Độ Ưu Tiên?

Tất cả các quy tắc CSS (dù nằm trong thẻ `<style>` hay file `.css` bên ngoài) đều có cùng mức độ ưu tiên dựa trên specificity và vị trí xuất hiện trong tài liệu HTML. Nếu hai quy tắc có cùng specificity, quy tắc nào xuất hiện sau sẽ ghi đè quy tắc trước.

**Ví Dụ 5: Internal CSS vs External CSS**

```

1 <!-- File index.html -->
2 <head>
3   <!-- Internal CSS viet TRUOC -->
4   <style>
5     p { color: red; }
6   </style>
7
8   <!-- External CSS duoc link SAU -->
9   <link rel="stylesheet" href="style.css">
10 </head>
11 <body>
12   <p>Doan van nay co mau gi?</p>
13 </body>

```

**BROWSER PREVIEW****Kết Quả Ví Dụ 5: Thẻ `<p>` Có Màu Xanh (Blue) Vì `style.css` Được Gọi Sau****Đoạn văn hiển thị màu blue**

Giải thích: Cả hai quy tắc `p` đều có specificity (0,0,0,1). Vì `style.css` được chèn sau thẻ `<style>`, quy tắc `p { color: blue; }` trong `style.css` sẽ ghi đè quy tắc trước đó.

[Bảng] Bảng Tóm Tắt Quy Tắc Đặc Biệt Trong Specificity

Quy tắc	Giải thích nguyên lý
!important không tính vào specificity	Nhưng nó ghi đè tất cả, trừ khi một quy tắc khác cũng có !important và có specificity cao hơn.
Universal selector (*) không ảnh hưởng	Vì nó không chọn phần tử cụ thể nào, điểm specificity bằng (0, 0, 0, 0).
Combinators (>, +, ~) không ảnh hưởng	Vì chúng chỉ định quan hệ giữa phần tử nhưng không làm tăng điểm specificity.
Internal và External CSS không ảnh hưởng	Độ ưu tiên được quyết định strictly bởi specificity và thứ tự xuất hiện trong tài liệu DOM.

1.7.5 5. Phân Tích Chi Tiết: Khi Nào Quy Tắc "Viết Sau Đè Viết Trước" Đúng Hoặc Sai?

5.1. Khi Nào Quy Tắc Này ĐÚNG?

[Khái Niệm] Điều Kiện Để Quy Tắc Source Order Áp Dụng

Nếu hai quy tắc CSS có **cùng điểm specificity** (và không có **!important**), quy tắc nào xuất hiện sau trong mã nguồn CSS sẽ được áp dụng.

●●● Ví Dụ 1: Hai Quy Tắc Có Cùng Specificity (0,0,0,1)

```

1 /* Viet truoc */
2 p { color: red; }
3
4 /* Viet SAU -> WINS! */
5 p { color: blue; }
```

●●● BROWSER PREVIEW

Kết Quả: Thẻ <p> Có Màu Xanh (Blue)

Đoạn văn có màu blue

Giải thích: Cả hai quy tắc p đều có specificity (0,0,0,1). Quy tắc p { color: blue; } xuất hiện sau nên ghi đè lên quy tắc trước.

5.2. Khi Nào Quy Tắc Này KHÔNG ĐÚNG Nữa? (3 Trường Hợp)

Có 3 trường hợp khiến quy tắc "viết sau đè viết trước" hoàn toàn không còn đúng:

[Cảnh Báo] Trường Hợp 1: Khi Một Quy Tắc Có !important

!important luôn ghi đè tất cả, trừ khi quy tắc khác cũng có **!important** và có specificity cao hơn. Dù quy tắc có **!important** viết trước, nó vẫn thắng quy tắc viết sau!

●●● Ví Dụ 2: !important Ghi Đè Quy Tắc Viết Sau

```
1 /* Viet TRUOC nhung co !important -> WINS! */
2 p { color: red !important; }
3
4 /* Viet SAU nhung khong co !important -> LOSES! */
5 p { color: blue; }
```

●●● BROWSER PREVIEW Kết Quả: Thẻ <p> Có Màu Đỏ (Red) Dù Quy Tắc Viết Sau Đặt Màu Blue**Đoạn văn có màu red**

Giải thích: Quy tắc p { color: red !important; } viết trước nhưng có !important nên chiến thắng hoàn toàn quy tắc p { color: blue; } viết sau.

[Cảnh Báo] Trường Hợp 2: Khi Một Quy Tắc Có Specificity Cao Hơn

Nếu một quy tắc có specificity cao hơn, nó sẽ ghi đè lên quy tắc có specificity thấp hơn, **dù xuất hiện trước hay sau!**

●●● Ví Dụ 3: Specificity Cao Hơn Ghi Đè Quy Tắc Viết Sau

```
1 /* Specificity: (0,0,0,1) - Viet TRUOC */
2 p { color: red; }
3
4 /* Specificity: (0,1,0,1) - Viet SAU nhưng specificity CAO HƠN -> WINS! */
5 #content p { color: blue; }
```

●●● BROWSER PREVIEW**Kết Quả: #content p (0,1,0,1) Thắng p (0,0,0,1)****Đoạn văn trong #content có màu blue**

Giải thích: #content p có specificity (0,1,0,1) cao hơn p (0,0,0,1). Dù bạn có đảo vị trí viết p sau #content p thì #content p vẫn luôn chiến thắng!

[Cảnh Báo] Trường Hợp 3: Khi CSS Nội Tuyển (Inline Styles) Được Sử Dụng

Inline styles (`style="..."`) có specificity cao nhất (1, 0, 0, 0), trừ khi bị **!important** ghi đè. Nó luôn ghi đè mọi quy tắc trong file CSS bên ngoài bất kể thứ tự.

●●● Ví Dụ 4: Inline Styles Ghi Đè CSS Bên Ngoài

```
1 <!-- Inline Style óc Specificity = (1,0,0,0) -->
2 <p style="color: green;">Đoạn văn này có màu gì?</p>
3
4 <style>
5   /* Specificity: (0,0,0,1) - Viet SAU nhưng thua Inline Style */
6   p { color: red; }
7 </style>
```

●●● BROWSER PREVIEW Kết Quả: Thẻ <p> Có Màu Xanh Lá (Green) Nhờ Inline Style**Đoạn văn hiển thị màu green**

Giải thích: `style="color: green;"` có specificity (1,0,0,0) cao hơn `p { color: red; }` (0,0,0,1), nên đoạn văn có màu green dù CSS bên ngoài đặt màu red.

[Bảng] Bảng Tổng Kết: Khi Nào Quy Tắc "Cái Nào Viết Sau Sẽ Áp Dụng" Đúng Hoặc Sai?

Trường hợp xét duyệt	Quy tắc sau được áp dụng?	Phân tích nguyên do
Hai quy tắc có cùng specificity, không có !important	ĐÚNG	Áp dụng nguyên tắc Source Order chuẩn.
Một quy tắc có !important , quy tắc kia không có	SAI	Quy tắc có !important luôn thắng.
Một quy tắc có specificity cao hơn	SAI	Quy tắc có specificity cao hơn luôn thắng.
Một quy tắc là Inline styles (<code>style="..."</code>)	SAI	Inline styles có điểm (1, 0, 0, 0) cao hơn.

1.7.6 6. Bài Tập & Ví Dụ Phân Tích Chuyên Sâu Độ Ưu Tiên**Ví Dụ Chuyên Sâu 1: Phân Tích Độ Ưu Tiên Danh Sách Trong **

Xét 2 bộ chọn đối lập nhắm vào danh sách:

Ví Dụ Danh Sách: li vs ul li

```

1 /* Specificity: (0,0,0,1) - 1 element */
2 li { color: blue; }
3
4 /* Specificity: (0,0,0,2) - 2 elements (ul + li) -> WINS! */
5 ul li { color: red; }

```

BROWSER PREVIEW**Kết Quả: Tất Cả Thẻ Đều Có Màu Đỏ (Red)****1. Mục danh sách thứ nhất (Red)****2. Mục danh sách thứ hai (Red)**

Giải thích: ul li có specificity (0,0,0,2) cao hơn li (0,0,0,1). Do đó tất cả thẻ bên trong sẽ hiển thị màu đỏ.

[Khái Niệm] Làm Thế Nào Để Đổi Sang Màu Xanh (Blue)? (3 Cách Xử Lý)

- Cách 1: Đổi thứ tự CSS?** Đảo `ul li { color: red; }` lên trước và `li { color: blue; }` xuống sau? **Vẫn có màu đỏ!** Vì `ul li` (0,0,0,2) > `li` (0,0,0,1), specificity cao hơn luôn thắng bất kể vị trí!
- Cách 2: Tăng specificity cho bộ chọn muốn đổi?** Viết `ul > li { color: blue; }`? Specificity của `ul > li` vẫn là (0,0,0,2) bằng với `ul li`. Nếu viết `ul > li` sau `ul li`, màu xanh sẽ thắng nhờ Source Order.
- Cách 3: Sử dụng !important?** Đặt `li { color: blue !important; }` sẽ ngay lập tức ghi đè màu đỏ thành màu xanh!

Ví Dụ Chuyên Sâu 2: Chuỗi 10 Cấp Độ Leo Thang Specificity Tăng Dần

Để hiểu thấu đáo bản chất thuật toán so sánh Specificity của trình duyệt, chúng ta cùng phân tích một ví dụ kinh điển với 10 quy tắc CSS cùng nhắm vào thẻ `<h2>`. Điểm Specificity sẽ leo thang liên tục từ 1 điểm đến 213 điểm qua từng cấp độ.

1. Cấu Trúc HTML Mẫu Thử Nghiệm

```

1 <div id="main">
2   <div id="head" class="head">
3     <h2 class="title">Specificity Test</h2>
4   </div>
5 </div>

```

●●● 2. Chuỗi 10 Quy Tắc CSS Leo Thang Specificity Tăng Dần

```

1  /* Cap 1: The don le (0,0,0,1) -> ~1 pt */
2  h2 { color: red; }
3
4  /* Cap 2: Class + The (0,0,1,1) -> ~11 pts */
5  h2.title { color: blue; }
6
7  /* Cap 3: Them div cha (0,0,1,2) -> ~12 pts */
8  div h2.title { color: orange; }
9
10 /* Cap 4: Them class .head (0,0,2,1) -> ~21 pts */
11 .head h2.title { color: green; }
12
13 /* Cap 5: Class cha + Tag div (0,0,2,2) -> ~22 pts */
14 div.head h2.title { color: gray; }
15
16 /* Cap 6: Them Pseudo-class :last-child (0,0,3,2) -> ~32 pts */
17 div.head h2.title:last-child { color: yellow; }
18
19 /* Cap 7: Them 1 ID #main (0,1,2,2) -> ~122 pts */
20 #main div.head h2.title { color: pink; }
21
22 /* Cap 8: Them ID thu 2 #head (0,2,1,2) -> ~212 pts */
23 #main div#head h2.title { color: purple; }
24
25 /* Cap 9: Pseudo-element chu dau (0,2,1,3) -> ~213 pts */
26 #main div#head h2.title::first-letter { color: red; }
27
28 /* Cap 10: Pseudo-element it class (0,2,0,3) -> ~203 pts */
29 #main div#head h2::first-letter { color: yellow; }

```

[Bảng] 3. Bảng Kết Quả Phân Tích 10 Cấp Độ Leo Thang Specificity

Cấp	Selector CSS	Specificity	Kết quả & Phân tích lý do
1	h2	(0,0,0,1)	Red – Bị Cấp 2 đè vì Cấp 2 có thêm Class .title .
2	h2.title	(0,0,1,1)	Blue – Bị Cấp 3 đè vì Cấp 3 có thêm thẻ div cha.
3	div h2.title	(0,0,1,2)	Orange – Bị Cấp 4 đè vì Cấp 4 có 2 Class: $C=2 > C=1$.
4	.head h2.title	(0,0,2,1)	Green – Bị Cấp 5 đè vì Cấp 5 có thêm thẻ div .
5	div.head h2.title	(0,0,2,2)	Gray – Bị Cấp 6 đè vì :last-child cộng thêm 1C.
6	div.head h2.title:last-child	(0,0,3,2)	Yellow – Bị Cấp 7 đè bẹp vì ID #main : $B=1 > B=0$.
7	#main div.head h2.title	(0,1,2,2)	Pink – Bị Cấp 8 đè vì thêm ID thứ 2 #head : $B=2$.
8	#main div#head h2.title	(0,2,1,2)	Purple [VÔ ĐỊCH H2] – Điểm cao nhất nhắm vào thân h2 !
9	#main div#head h2.title::first-letter	(0,2,1,3)	Red [VÔ ĐỊCH CHỮ 'S'] – Nhắm vào ::first-letter , thắng Cấp 10.
10	#main div#head h2::first-letter	(0,2,0,3)	Yellow – Thua Cấp 9 vì thiếu Class .title : $C=0 < C=1$.

[Khái Niệm] 4. Giải Đáp 3 Thắc Mắc Cốt Lõi Về Thuật Toán So Sánh**1. Tại sao Cấp 4 (0,0,2,1) màu Green lại thắng Cấp 3 (0,0,1,2) màu Orange?**

Khi so sánh hai điểm số (0, 0, 2, 1) và (0, 0, 1, 2), trình duyệt so sánh từ trái sang phải: Nhóm A bằng nhau (0=0), Nhóm B bằng nhau (0=0), đến Nhóm C: Cấp 4 có 2 Class ($C = 2$), Cấp 3 chỉ có 1 Class ($C = 1$). Vì $2 > 1$, Cấp 4 chiến thắng **mặc dù Cấp 3 có nhiều thẻ HTML hơn!**

2. Tại sao Cấp 7 (0,1,2,2) màu Pink lại đánh bại Cấp 6 (0,0,3,2) màu Yellow?

Cấp 6 dù gom tới 3 Class/Pseudo-class (điểm $C = 3$), nhưng Cấp 7 xuất hiện 1 bộ chọn ID **#main** (điểm $B = 1$). Vì nhóm ID (B) nằm ở vị trí ưu tiên cao hơn nhóm Class (C), nên $B = 1$ lập tức đè bẹp $B = 0$ bất kể Cấp 6 có bao nhiêu Class đi nữa!

3. Tại sao chữ 'S' lại có màu Red còn toàn bộ chữ 'pecificity Test' lại màu Purple?

- Cấp 8 nhắm vào toàn bộ thẻ **<h2>** và đạt điểm Specificity (0, 2, 1, 2) cao nhất → Tô màu **Purple** cho thân thẻ.
- Cấp 9 dùng pseudo-element **::first-letter** nhắm riêng vào chữ cái đầu tiên **'S'** với điểm (0, 2, 1, 3) thắng Cấp 10 (0, 2, 0, 3) → Tô màu **Red** riêng cho chữ 'S'.



Specificity Test

Chữ 'S' màu Đỏ (Red) [Quy tắc 9 thắng]

Chữ 'pecificity Test' màu Tím (Purple) [Quy tắc 8 thắng]

1.7.7 7. Pseudo-class :not(), :is() Và :where() Trong CSS Modern

7.1. Pseudo-class :not() – Bản Thân Không Tính Điểm

[Khái Niệm] Quy Tắc Đặc Biệt Của :not()

- Bản thân **:not()** KHÔNG được tính điểm specificity. Nó là một khung vô hình.
- Chỉ có nội dung bên trong ngoặc đơn mới được tính. Ví dụ: **:not(.active)** cộng 1 class (**.active**) vào nhóm C.
- Kết quả: **div:not(.active)** có specificity = **(0,0,1,1)** – gồm 1 thẻ **div** (D) + 1 class **.active** (C).

7.2. Pseudo-class :is() – Lấy Specificity Của Đối Số Cao Nhất

[Khái Niệm] Quy Tắc Của :is()

:is() gom nhóm selector và lấy specificity của **đối số có điểm CAO NHẤT** trong danh sách.
 Ví dụ: **:is(.btn, #submit)** lấy specificity của **#submit** (0, 1, 0, 0).

7.3. Pseudo-class :where() – Specificity Luôn Bằng 0

[Khái Niệm] Quy Tắc Của :where() – Zero Specificity

:where() luôn có **specificity = (0,0,0,0)**, bất kể nội dung bên trong là gì. Rất hữu ích để viết CSS mặc định trong thư viện để ghi đè.

[Bảng] Bảng So Sánh Nhanh :not() vs :is() vs :where()

Pseudo-class	Cách tính Specificity	Ví dụ	Specificity kết quả
:not(.active)	Lấy specificity của đối số bên trong	div:not(.active)	(0,0,1,1)
:is(.btn, #id)	Lấy specificity của đối số cao nhất	:is(.btn, #submit)	(0,1,0,0)
:where(.btn, #id)	Luôn = 0 , bất kể nội dung	:where(#main, .box)	(0,0,0,0)

1.7.8 8. Sơ Đồ Tính Điểm Specificity Từng Bước (Step-by-Step)

Ví dụ minh họa: Tính specificity cho `div#main .btn:hover`

[Bảng] Sơ Đồ Tính Điểm Từng Bước Cho Selector: `div#main .btn:hover`

Bước	Thành phần	Phân loại	Nhóm	Cộng điểm
1	div	Thẻ HTML (Element)	D	+1 vào D
2	#main	ID Selector	B	+1 vào B
3	.btn	Class Selector	C	+1 vào C
4	:hover	Pseudo-class	C	+1 vào C
Tổng cộng:			A=0, B=1, C=2, D=1	
Specificity:			(0, 1, 2, 1)	

1.7.9 9. Kinh Nghiệm Thực Tế & Tối Ưu Kiến Trúc CSS

[Bảng] Bảng Quy Tắc "Nên & Không Nên" Trong Thực Tế Lập Trình Frontend

[KHÔNG NÊN] làm trong CSS	[NÊN] thực hiện (Best Practices)
Dùng ID làm selector chính trong CSS (<code>#header { background: blue; }</code>) làm code bị cứng, khó ghi đè.	Sử dụng Class cho tất cả các bộ chọn (<code>.header { background: blue; }</code>) để linh hoạt và tái sử dụng.
Lạm dụng <code>!important</code> để giải quyết xung đột CSS tạm thời, gây rác mã nguồn.	Chỉ dùng <code>!important</code> cho các utility class đặc biệt (như <code>.d-none</code>) hoặc override thư viện ngoài.
Lồng selector quá sâu: <code>body div.container section.main article.post p</code> gây tăng điểm specificity không cần thiết.	Viết selector ngắn gọn, phẳng (Flat Selectors): <code>.post p</code> hoặc áp dụng phương pháp luận đặt tên BEM (<code>.post__text</code>).

[Bảng] Bảng Mẹo Nhớ Nhanh Sức Mạnh Của Các Loại Selector (Memory Tips)

Mức độ Selector	Mẹo ghi nhớ dễ thuộc lòng
!important	Nói là làm – Luôn luôn chiến thắng mọi bộ chọn khác.
Inline Style (1,0,0,0)	Gần nhất phần tử – Viết trực tiếp trong thuộc tính <code>style="..."</code> trên thẻ HTML.
ID Selector (0,1,0,0)	Định danh duy nhất – Rất mạnh mẽ, định danh duy nhất cho phần tử.
Class / Attr / Pseudo-class (0,0,1,0)	Chung nhóm – Ổn định, linh hoạt, được sử dụng phổ biến nhất trong thực tế.
Element / Pseudo-element (0,0,0,1)	Thẻ HTML – Mức độ yếu nhất trong các loại bộ chọn trực tiếp.
Inheritance / Kế thừa (0,0,0,0)	Mặc định – Dễ bị bất kỳ bộ chọn trực tiếp nào ghi đè.

[Khái Niệm] Tổng Kết Quy Tắc Vàng Về Thứ Tự Ưu Tiên CSS

Công thức tổng quát: `Specificity = (A, B, C, D)` hoặc `Specificity = (a, b, c, d)`

Chuỗi thứ tự ưu tiên tuyệt đối từ cao xuống thấp (1 → 6):

1. **!important trong CSS** – Cấp quyền ưu tiên tuyệt đối (ghi đè mọi bộ chọn).
2. **Style nội tuyến (Inline style) (1,0,0,0)** – Đặt trực tiếp trong thuộc tính `style="..."`.
3. **Bộ chọn ID (#id) (0,1,0,0)** – Mức ưu tiên cao cho định danh duy nhất.
4. **Class, Attribute, Pseudo-class (0,0,1,0)** – Nhóm bộ chọn phổ biến (`.class`, `[attr]`, `:hover`).
5. **Thẻ HTML & Pseudo-element (0,0,0,1)** – Nhóm thẻ phần tử (`div`, `p`, `::before`).
6. **Kế thừa & Universal Selector (0,0,0,0)** – Kế thừa mặc định và bộ chọn chung (`*`) – *Yếu nhất*.

1.8 Thuộc Tính Opacity (Độ Mờ & Độ Trong Suốt Trong CSS)

Thuộc tính `opacity` trong CSS cho phép lập trình viên kiểm soát mức độ hiển thị, độ mờ và độ trong suốt của bất kỳ phần tử HTML nào trên giao diện web.

1. Opacity Là Gì?

[Khái Niệm] Khái Niệm & Giá Trị Của Thuộc Tính Opacity

Opacity xác định độ trong suốt của một phần tử HTML. Nó quyết định xem ánh sáng/màu nền phía sau phần tử có thể nhìn xuyên qua được hay không.

Dải giá trị chấp nhận từ 0 đến 1:

- **0** → Hoàn toàn trong suốt (phần tử vô hình nhưng vẫn chiếm vị trí không gian trên luồng tài liệu).
- **1** → Không trong suốt (hiển thị rõ ràng 100% như bình thường).
- **Giữa 0 và 1 (ví dụ 0.1 – 0.9)** → Mờ dần theo tỷ lệ phần trăm (ví dụ: **0.5** mờ 50%, **0.2** mờ 80%).

2. Cú Pháp Khai Báo CSS:

●●● Cú Pháp Định Mức Độ Mờ Opacity

```
1 selector {
2     opacity: value; /* Giá trị số từ 0.0 đến 1.0 */
3 }
```

- **0**: Hoàn toàn trong suốt (ẩn hoàn toàn).
- **1**: Hoàn toàn không trong suốt (hiển thị rõ ràng).
- **Giữa 0 và 1**: Mờ dần theo mức độ (ví dụ: **0.5** là mờ 50%).

3. Ví Dụ Minh Họa & Kết Quả Trực Quan Trên Trình Duyệt:

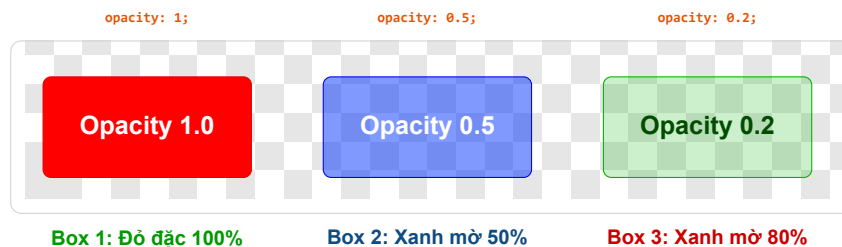
●●● Khai Báo 3 Mức Độ Mờ Cho Các Khối Div

```

1 <div class="box full-opacity">Opacity: 1</div>
2 <div class="box semi-opacity">Opacity: 0.5</div>
3 <div class="box low-opacity">Opacity: 0.2</div>
4
5 <style>
6 .box {
7     width: 200px;
8     height: 100px;
9     margin: 10px;
10    padding: 10px;
11    color: white;
12    text-align: center;
13    line-height: 100px;
14    font-size: 20px;
15 }
16 .full-opacity {
17     background: #ff0000;
18     opacity: 1; /* Không mờ */
19 }
20 .semi-opacity {
21     background: #007bff;
22     opacity: 0.5; /* Mờ 50% */
23 }
24 .low-opacity {
25     background: #28a745;
26     opacity: 0.2; /* Mờ 80% */
27 }
28 </style>

```

●●● BROWSER PREVIEW Kết Quả Trực Quan 3 Khối Box Trên Nền Ca-rô Trong Suốt Mờ Trình Duyệt



4. Lưu Ý Quan Trọng Khi Sử Dụng Opacity:

[Cảnh Báo] 3 Cạm Bẫy Quan Trọng Cần Ghi Nhớ Với Opacity

1. **Opacity làm mờ CẢ phần tử lẫn tất cả nội dung con bên trong:** Khi bạn áp dụng **opacity** cho một thẻ cha, toàn bộ chữ, hình ảnh, nút bấm con bên trong thẻ cha đó **đều bị mờ theo**.

●●● Lỗi Tự Động Mờ Nội Dung Con

```
1 .parent {
2   background: purple;
3   opacity: 0.5; /* Chu và nút bấm con bên trong cũng bị mờ 50%! */
4 }
```

2. **Phần tử vẫn chiếm không gian luồng tài liệu dù opacity: 0:** Phần tử không bị xóa khỏi bố cục trang mà chỉ bị biến thành vô hình. (Nếu muốn ẩn hoàn toàn và xóa khỏi luồng bố cục, hãy dùng **display: none;**).
3. **Không thể click tương tác khi phần tử vô hình (opacity: 0):** Mặc dù phần tử vẫn tồn tại nhưng người dùng không thể nhìn thấy để thao tác (tương tự như bị vô hình).

5. Thay Thế Bằng RGBA Để Chỉ Làm Mờ Nền (Giữ Chữ Rõ Nét):

Nếu bạn muốn **chỉ làm mờ màu nền** nhưng vẫn giữ nội dung văn bản và phần tử con hiển thị rõ nét 100%, hãy sử dụng giá trị màu **rgba()** thay vì thuộc tính **opacity**:

●●● Sử Dụng RGBA Để Chỉ Mờ Màu Nền

```
1 .box {
2   background: rgba(255, 0, 0, 0.5); /* Nền đỏ mờ 50%, văn bản bên trong vẫn rõ
3   100% */
3 }
```

[Khái Niệm] Cấu Trúc Giá Trị Màu RGBA (Red, Green, Blue, Alpha)

RGBA (Red, Green, Blue, Alpha):

- **R:** Giá trị kênh Đỏ (Red: 0 → 255).
- **G:** Giá trị kênh Xanh lá (Green: 0 → 255).
- **B:** Giá trị kênh Xanh dương (Blue: 0 → 255).
- **A:** Kênh Alpha độ trong suốt (0 → 1).

Ưu điểm vượt trội của RGBA: Chỉ làm mờ riêng màu nền, nội dung văn bản và các phần tử con bên trong **hoàn toàn không bị ảnh hưởng!**

6. Các Ứng Dụng Thực Tế Phổ Biến Của Opacity:

[Khái Niệm] 4 Kịch Bản Ứng Dụng Opacity Trong Dự Án Web Thực Tế

1. **Hiệu ứng di chuột (Hover Effect):** Làm mờ nhẹ thẻ hoặc ảnh khi di chuột qua để báo hiệu khả năng nhấp chuột:

●●● Làm Mờ Khi Di Chuột Di Vào Card

```
1 .card:hover {
2     opacity: 0.7;
3 }
```

2. **Màn hình phủ tải trang (Loading Screen Overlay):** Làm mờ tối toàn bộ trang web khi hiển thị cửa sổ tải dữ liệu:

●●● Tạo Màn Phủ Tối Overlay

```
1 .overlay {
2     position: fixed;
3     top: 0;
4     left: 0;
5     width: 100%;
6     height: 100%;
7     background: rgba(0, 0, 0, 0.5);
8 }
```

3. **Hiệu ứng ảnh mờ dần / Hiện dần (CSS Fade Animation):**

●●● Tạo Animation Hiện Dần Vừa Mờ Đến Rõ

```
1 @keyframes fade {
2     0% { opacity: 0; }
3     100% { opacity: 1; }
4 }
```

4. **Làm nổi bật & Vô hiệu hóa nút bấm (Disabled State):** Kết hợp `opacity` với `pointer-events: none` để vừa làm mờ vừa khóa tương tác nhấp chuột:

●●● Vô Hiệu Hóa Thao Tác Click Vẫn Làm Mờ Nut

```
1 .disabled {
2     opacity: 0.5;
3     pointer-events: none; /* Vô hiệu hóa mọi tương tác click */
4 }
```

7. Bảng Tổng Kết Trực Quan Kiến Thức Opacity:

[Bảng] Bảng Tổng Kết Tra Cứu Toàn Diện Về Thuộc Tính Opacity & Giải Pháp RGBA		
Thuộc tính / Giá trị	Ý nghĩa & Hành vi hiển thị	Cú pháp ví dụ minh họa
opacity: 1	Phần tử hoàn toàn hiển thị (không mờ).	opacity: 1;
opacity: 0.5	Phần tử và mọi thể con mờ 50%.	opacity: 0.5;
opacity: 0	Phần tử hoàn toàn trong suốt (vẫn chiếm bố cục).	opacity: 0;
pointer-events	Kết hợp với opacity để vô hiệu hóa click chuột.	pointer-events: none;
rgba(...)	Chỉ làm mờ màu nền, giữ nguyên chữ con 100%.	background: rgba(0, 0, 0, 0.5);

[Khái Niệm] Tóm Lại Cốt Lõi Kiến Thức

- **opacity** là công cụ cực kỳ mạnh mẽ để tạo hiệu ứng mờ, lớp phủ overlay, và làm nổi bật nội dung giao diện.
- Thuộc tính này tác động đến **cả phần tử cha lẫn toàn bộ phần tử con**. Nếu chỉ muốn mờ màu nền, hãy sử dụng **rgba()** hoặc kết hợp **pointer-events** để kiểm soát trải nghiệm người dùng tốt nhất.

Mờ Rộng Kiến Thức & Kinh Nghiệm Thực Tế Chuyên Sâu Về Opacity

[Khái Niệm] 1. Nguyên Lý Tạo Stacking Context Trực Tiếp Từ Opacity < 1

Trong cơ chế render của trình duyệt, khi một phần tử có thuộc tính **opacity < 1**, trình duyệt tự động tạo ra một **Ngữ cảnh xếp chồng mới (Stacking Context)** trên phần tử đó!

Hệ quả quan trọng: Thẻ con chứa **z-index: 9999** bên trong phần tử có **opacity: 0.99** sẽ **KHÔNG THỂ** ngoi lên trên các phần tử bên ngoài có z-index cao hơn thẻ cha! Đây là nguyên nhân khiến nhiều lập trình viên Junior loay hoay không hiểu tại sao z-index của thẻ con bị vô hiệu hóa.

[Bảng] So Sánh 3 Phương Pháp Ẩn Phần Tử HTML Trong CSS (*opacity vs visibility vs display*)

Tiêu chí so sánh	opacity: 0;	visibility: hidden;	display: none;
Chiếm không gian layout?	CÓ (Vẫn chiếm diện tích).	CÓ (Vẫn chiếm diện tích).	KHÔNG (Xóa khỏi luồng layout).
Click / Sự kiện DOM?	Vẫn nhận click ngoại trừ khi dùng pointer-events: none.	KHÔNG nhận sự kiện mouse/click.	KHÔNG nhận bất kỳ sự kiện nào.
Hỗ trợ CSS Transition / Animation?	CÓ (Mượt mà từ 0 → 1).	Hỗ trợ hạn chế (chỉ bật/tắt ở cuối).	KHÔNG (Biến mất tức thì, không transition).
Tăng tốc phần cứng GPU?	CÓ (Dùng GPU Composite Layer).	KHÔNG.	KHÔNG.

[Mẹo] 2. Tối Ưu Hiệu Năng Animation Bằng Tăng Tốc Phần Cứng GPU

opacity cùng với **transform** là hai thuộc tính CSS duy nhất được xử lý trực tiếp trên tầng **Composite Layer** của GPU (Graphics Processing Unit). Khi thay đổi **opacity**, trình duyệt **KHÔNG cần phải tính toán lại Layout (Reflow) hay vẽ lại pixel (Repaint)**, giúp hiệu ứng đạt mượt mà chuẩn **60 FPS** ngay cả trên thiết bị di động cấu hình yếu.

[Cảnh Báo] 3. Bộ Câu Hỏi Phỏng Vấn Senior Frontend Về Opacity

- Câu hỏi 1: Làm thế nào để thẻ cha mờ nhưng thẻ con bên trong vẫn giữ nguyên opacity 100%?**
→ **Trả lời:** Không thể đặt **opacity** trên thẻ cha rồi chỉnh thẻ con **opacity: 1** vì opacity mang tính chất nhân dồn ($0.5 \times 1.0 = 0.5$). Giải pháp là dùng màu **rgba()** cho nền thẻ cha, hoặc tách thẻ con ra thành phần tử đồng cấp rồi xếp chồng bằng CSS **position: absolute**.
- Câu hỏi 2: Tại sao khi đổi opacity từ 1 về 0 lại không click được nếu dùng pointer-events: none?**
→ **Trả lời:** Thuộc tính **pointer-events: none** giúp phần tử hoàn toàn bỏ qua mọi sự kiện chuột, giúp các sự kiện click xuyên qua phần tử vô hình để tác động tới phần tử bên dưới.
- Câu hỏi 3: Sự khác biệt bản chất giữa opacity: 0 và display: none trong SEO và Accessibility (SEO & Màn hình đọc Screen Reader)?**
→ **Trả lời:** **display: none** ẩn hoàn toàn với cả người dùng lẫn Màn hình đọc (**Screen Reader**). Trong khi **opacity: 0** vẫn được trình đọc màn hình đọc dữ liệu văn bản bên trong cho người khiếm thị.

Bố Cục Modern CSS: Flexbox & CSS Grid

2.1 Bố Cục Một Chiều Với CSS Flexbox

Các thuộc tính Container: `display: flex`, `flex-direction`, `justify-content`, `align-items`, `gap`. Thuộc tính Flex Items: `flex-grow`, `flex-shrink`, `flex-basis`.

2.2 Bố Cục Hai Chiều Với CSS Grid

Khai báo lưới `grid-template-columns`, `grid-template-rows`, kỹ thuật `repeat(auto-fit, minmax(250px, 1fr))` giúp responsive tự động không cần Media Query.

Responsive Web Design Nâng Cao

3.1 Tư Duy Mobile-First

Xây dựng giao diện từ màn hình nhỏ nhất (Mobile), mở rộng dần sang Tablet và Desktop để tối ưu hiệu năng.

3.2 Fluid Typography & Container Queries

Sử dụng hàm CSS `clamp(1rem, 2.5vw, 2rem)` cho cỡ chữ linh hoạt và tính năng hiện đại **Container Queries**.

CSS Variables & BEM Architecture

4.1 CSS Custom Properties (CSS Variables)

Khai báo biến toán cục trong `:root` để làm Dark Mode và Design System.

4.2 Phương Pháp Đặt Tên BEM (Block - Element - Modifier)

Tổ chức CSS sạch sẽ, tránh xung đột tên lớp bằng BEM (ví dụ: `.card__title--active`).

CSS Animations & Transitions

5.1 Hiệu Ứng Chuyển Động Với CSS Transitions

Thuộc tính `transition`, `transition-timing-function` (`ease-in-out`, `cubic-bezier`).

5.2 Keyframe Animations

Viết animation phức tạp bằng `@keyframes` và biến đổi không gian 2D/3D `transform`.